ReUseAI

Dossier de lancement — M1 MIAGE Apprentissage — v0.1.0-SNAPSHOT

Jorys KANCEL, SALEKHA FALL, SARAH METREF, CAMILIA SABA, MOHAMED GUEYE · 20 octobre 2026

01 Équipe, comptes développeurs et dépôt

MEMBRE	COMPTE GITHUB	EMAIL DES COMMITS (`GIT CONFIG USER.EMAIL`)
Jorys KANCEL	`@Sinie-Kun`	À compléter
SALEKHA FALL	`@salekhaf`	À compléter
SARAH METREF	`@SarahMETREF`	À compléter
CAMILIA SABA	`@fromwonderland`	À compléter
MOHAMED GUEYE	`@Moh-svg04`	À compléter

Chaque membre vérifiera que son email Git est associé à son compte GitHub, pour que ses commits lui soient correctement attribués. **Dépôt** : https://github.com/Moh-svg04/reUseAi-projet-m1-miage · **page de projet** : https://Moh-svg04.github.io/reUseAi-projet-m1-miage/.

02 Projet et caractère innovant

ReUseAI est une application web d'aide à la décision destinée aux associations et aux équipes de collecte qui organisent le réemploi d'objets. Elle vise à coordonner des campagnes de collecte à domicile, à prendre en compte les capacités des véhicules et à aider à répartir les objets récupérés entre les associations selon leurs besoins.

Le problème traité ne consiste pas seulement à enregistrer des objets : il faut choisir quels lots collecter, regrouper les adresses, construire des chargements compatibles avec les capacités en poids et en volume, planifier les tournées et réagir aux imprévus. Une organisation manuelle devient difficile lorsque plusieurs contraintes doivent être respectées simultanément.

L'entité pivot est le **lot d'objets**, rattaché à une campagne et caractérisé notamment par sa catégorie, son état, son poids, son volume, sa localisation et son statut. Les campagnes, véhicules, équipes, besoins des associations, tournées et affectations complètent le modèle métier.

Le caractère innovant de ReUseAI repose sur l'intégration d'algorithmes explicables dans un parcours applicatif complet : priorisation multicritère, regroupement géographique, chargement sous contraintes, optimisation de tournées et affectation des objets. Les propositions seront présentées aux responsables, qui pourront les examiner et les valider. Les résultats seront évalués au moyen de contraintes vérifiables et comparés, lorsque cela est pertinent, à des solutions de référence simples.

Le géocodage sera initialement renseigné manuellement. Le calcul des distances routières s'appuiera sur un service de routage compatible avec les données OpenStreetMap. Les appels externes seront isolés derrière un adaptateur et les tests utiliseront des données déterministes afin de rester reproductibles.

03 Scénarios d'usage

- **En tant que responsable de campagne**, je veux classer les lots à collecter selon des critères pondérés, afin de décider quelles collectes traiter en priorité.
- **En tant que responsable logistique**, je veux regrouper les adresses proches et constituer des chargements respectant le poids et le volume autorisés, afin de préparer des collectes réalisables.
- **En tant que membre d'une équipe de collecte**, je veux consulter une tournée ordonnée selon les distances routières, afin de limiter les déplacements inutiles.
- **En tant que responsable d'association**, je veux obtenir une proposition de répartition des objets en fonction des besoins et des stocks disponibles, afin de mieux répondre aux demandes des bénéficiaires.
- **En tant que responsable logistique**, je veux recalculer un plan lorsqu'un véhicule ou un point de collecte devient indisponible, afin de limiter les conséquences de l'imprévu.
- **En tant qu'administrateur**, je veux consulter les contraintes violées et les indicateurs d'un plan, afin de vérifier sa faisabilité avant de le valider.

04 Plan des features

Une feature a un effet sur **la BD, le back, l'API REST et le front**. Chaque feature est portée par **une seule personne** et repose sur **un algorithme identifié**. La répartition ci-dessous est une proposition initiale tournante : elle devra être validée par l'équipe et chaque fiche devra préciser les contraintes, les cas limites et la complexité de l'algorithme réellement implémenté.

VERSION	JORYS KANCEL	SALEKHA FALL	SARAH METREF	CAMILIA SABA	MOHAMED GUEYE
v0.1.0 — 24/11	Priorisation des lots — score multicritère pondéré	Regroupement géographique — DBSCAN	Chargements sous contraintes — bin packing multidimensionnel heuristique	Affectation initiale objets-besoins — graphe biparti / flot	Planification des créneaux — ordonnancement glouton sous contraintes
v0.2.0 — 08/12	Optimisation des tournées — tournée initiale puis amélioration 2-opt	Affectation des tournées aux équipes — affectation pondérée	Réorganisation après imprévu — réoptimisation locale	Répartition globale du stock — flot de coût minimum	Équilibrage des charges de travail — recherche locale par échanges
v1.0.0 — 19/01	Comparaison de plans — dominance de Pareto	Sélection des objets à réparer — programmation dynamique type sac à dos	Détection de lots atypiques — méthode statistique IQR	Estimation des besoins — moyenne mobile pondérée	Évaluation de la qualité d'un plan — vérification des contraintes et calcul d'indicateurs

Les complexités seront documentées dans les fiches de features en fonction des structures de données et de l'implémentation choisies. Les fonctionnalités dépendant de données historiques seront évaluées avec prudence : les jeux synthétiques permettent de

tester l'algorithme, mais ne suffisent pas à revendiquer une précision prédictive réelle. La distinction entre fonctionnalités devra être justifiée par des objectifs et contraintes différents, notamment pour l'affectation initiale et la répartition globale, ainsi que pour la comparaison et la validation des plans.

05 Architecture

```
Front  --HTTP/JSON-->  API REST  -->  Back (cas d'utilisation)  --JPA-->  BD
(web)                (Spring MVC)      (Java 21, hexagonal)           (PostgreSQL)
```

COUCHE	CHOIX	REMARQUES
Front	application web : HTML, CSS, JavaScript natif	client de l'API REST uniquement ; dossier `front/`
API REST	Spring Boot 4 (Spring MVC), OpenAPI / Swagger UI	versionnée sous `/api/v1`, erreurs au format RFC 9457
Back	Java 21, Spring Boot 4, Spring Data JPA, Bean Validation	**architecture hexagonale**
BD	PostgreSQL 17 (H2 pour les tests)	schéma versionné par **Flyway**, une migration par feature
Qualité	JUnit 5, AssertJ, ArchUnit, JaCoCo, Codecov, SonarCloud	intégration continue GitHub Actions
Conteneurs	Docker Compose	BD dès v0.1.0, application en v0.2.0

Le modèle métier commun s'organise autour du lot d'objets, lié à une campagne, une localisation, un statut de collecte et, après récupération, à une affectation éventuelle. Les véhicules portent des capacités en poids et en volume ; les associations expriment des besoins par catégorie et quantité. Les migrations Flyway versionnent les évolutions du schéma.

Qualité du code. Au-delà de l'architecture hexagonale :

- **Clean Code** : noms métier explicites, fonctions courtes, pas de duplication, commentaires qui expliquent le pourquoi ;
- **SOLID**, objets valeur immuables et invariants vérifiés à la construction ;
- **design patterns** quand ils servent, nommés dans les fiches de features ;
- conventions **REST** et règles de sécurité **OWASP** ;
- quality gate SonarCloud au vert.

Les algorithmes seront placés dans le domaine Java, indépendamment de Spring et des services externes. Les appels au service de routage seront encapsulés derrière un port et un adaptateur. Les tests utiliseront des données synthétiques reproductibles et des réponses simulées pour ne pas dépendre d'un service externe. Les secrets de configuration ne seront pas versionnés.

06 Démarche pour chaque feature

Les étapes s'enchaînent **dans cet ordre, sans en sauter** : on ne commence une étape que lorsque la précédente est terminée.

ÉTAPE	CE QU'ON FAIT	TERMINÉ QUAND
1. Idéation	Besoin et scénario ; **algorithme retenu**, complexité, alternatives ; impact sur le schéma et l'API ; critères d'acceptation. Issue + fiche `doc/features/` .	La fiche est validée par un autre membre.
2. Back	Migration Flyway ; **domaine** en Java pur (modèle, règles, algorithme) ; **application** (ports, service) ; adaptateur de persistance.	Le code compile et ArchUnit passe.
3. Tests	Unitaires de l'algorithme (cas limites, invariants, propriétés) ; services avec faux adaptateurs ; intégration de la persistance.	`./gradlew build` passe ; chaque critère d'acceptation est couvert.
4. API REST	Endpoints `/api/v1` , DTO validés, erreurs `problem+json` , OpenAPI ; tests de l'API.	Visible dans Swagger UI ; tests de l'API verts.
5. Front	Écrans qui consomment **uniquement** l'API.	Scénario jouable de bout en bout ; PR relue ; fiche complétée avec son araignée.

07 Protocole d'usage de l'IA

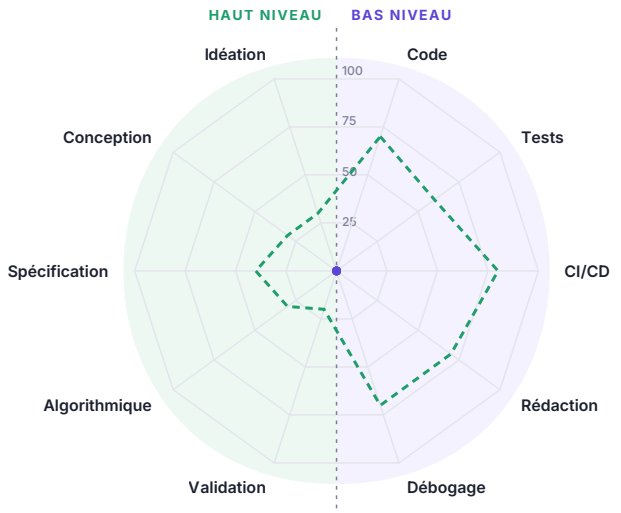
Principe : l'IA assiste, l'humain reste l'auteur. On ne laisse jamais l'IA tout faire sans comprendre : tout ce qui est commité a été relu et compris par son auteur, capable de l'expliquer, de le modifier et de le refaire sans aide. Les consignes des assistants de code sont versionnées dans `CLAUDE.md` .

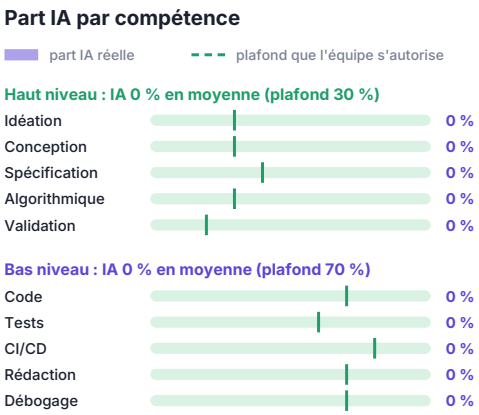
ACTIVITÉ	CE QUE FAIT L'IA	CE QUE FAIT L'HUMAIN
Idéation et conception	Critique les idées, propose des options	Choisit, justifie, rédige la fiche
Algorithme	Explique, relit, signale les cas oubliés	**Conçoit et écrit le cœur de l'algorithme**
Code technique	Peut générer un premier jet	Relit chaque ligne, adapte, assume
Tests	Suggère des cas	Définit cas limites et propriétés ; vérifie qu'un test échoue quand on casse le code
CI/CD, conteneurisation	Peut générer workflows et Dockerfile	Comprend chaque étape, vérifie l'exécution, gère les secrets
Documentation	Aide à rédiger et mettre en forme	Valide le contenu, vérifie les faits

Interdits. Commiter du code qu'on ne comprend pas ; confier des secrets ou des données personnelles à l'IA ; commiter sous une autre identité ; présenter comme humain un travail de l'IA.

Traçabilité. Section « Usage de l'IA » dans chaque pull request. Part de l'IA déclarée **par compétence** (haut niveau / bas niveau) dans un graphe en araignée, comparée à un plafond que l'équipe s'autorise (annexe). Les déclarations décriront le travail réellement effectué, sans valeurs fictives.

08 Annexe : part IA / part humaine





Valeurs dans `doc/part-ia.json`, régénérées à chaque compilation.